

SN-IES-1-004-RTE Feedback API – Description and Integration

2020-11-04

Support Note SN-IES-1-004

Author(s) Ragavendran, Murugaiyan
Restrictions Public Document

Table of Contents

1	Overview	2
2	What is the Rte_Feedback API?	2
3	Is the Rte_Feedback API supported in explicit and implicit sender receiver communication?	2
4	Can the Rte_Feedback API contain wait points?	2
5	Can the Rte_Feedback functionality trigger a runnable?	2
6	What are the return values from the Rte_Feedback API?	2
7	How to configure the RTE_Feedback API?	3
	7.1 Port Configuration:	3
	7.2 Runnable Configuration for Wait Point:	4
	7.3 Different Runnable for Feedback API:	4
	7.4 COM Configuration (Inter ECU Communication)	5
8	The Rte Feedback State gets stuck at RTE_E_NO_DATA – is this an Error?	5
9	Contacts	6

1 Overview

This Support Note answers FAQs in the field of **Rte_Feedback API**.

2 What is the Rte_Feedback API?

It is used to retrieve the transmission acknowledgment from the lower layer i.e. COM (in Inter ECU Communication) or other SWCs (in Intra ECU Communication). With the help of the **Rte_Feedback API** the SWCs can check if the current data was already transmitted or if any problem occurred during transmission.

3 Is the Rte_Feedback API supported in explicit and implicit sender receiver communication?

The **Rte_Feedback API** is only available in sender receiver communication and it is supported in both explicit and implicit sender receiver communication.

1. In explicit communication it is called **Rte_Feedback**.
2. In implicit communication it is called **Rte_iFeedback**. But currently this is not supported by the Vector MICROSAR RTE.

4 Can the Rte_Feedback API contain wait points?

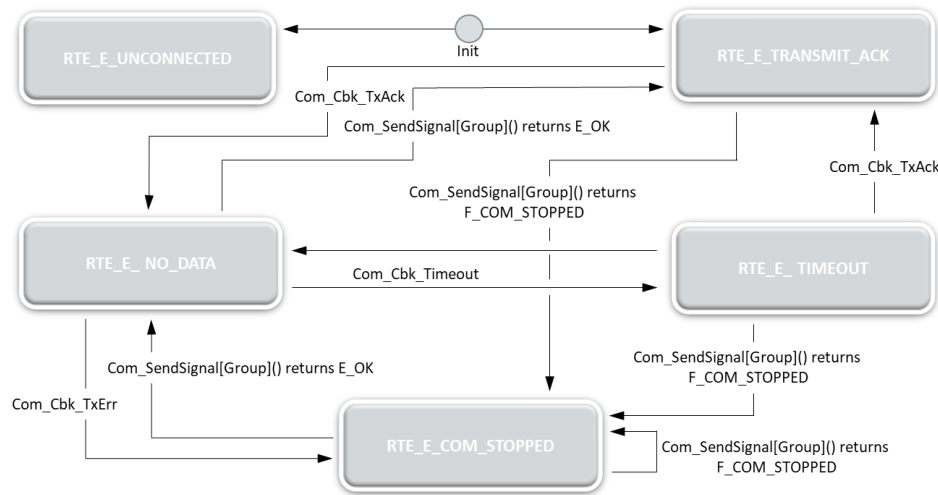
The **Rte_Feedback API** can be configured with wait points and timeout values, then the RTE will wait in the **Rte_Feedback API** until the configured timeout expires. Once the Rte timeout expires without receiving notification from the receiver (COM/Receiver) the **Rte_Feedback API** returns **RTE_FBK_NO_DATA**.

5 Can the Rte_Feedback functionality trigger a runnable?

Yes, the **Rte_Feedback** functionality can be configured to trigger a runnable (other than **Rte_Write API**). In this case the runnable should be triggered with the event “data send completed”. When Rte receives the **Data send completion** notification from COM Tx callback or the receiver, then the Rte will trigger the configured runnable.

6 What are the return values from the Rte_Feedback API?

There are 4 different return values provided by the **Rte_Feedback API**.



> RTE_FBK_COM_STOPPED:

This is used only in the inter ECU communication.

- > Either when COM (**Com_SendSignal/Com_SendSignalGroup...**) has rejected the last transmission request from the **Rte_Write** or **Rte_Send** API, then the **Rte_Feedback API** returns with an **RTE_FBK_COM_STOPPED**.
- > Or when the transmission request from Rte (**Com_SendSignal...()**) was accepted (return value **E_OK**) but the Com Ipd Group where the corresponding Tx-Signal-containing PDU is inherited is switched off before the lower layer was able to complete the transmission.

> RTE_FBK_TIMEOUT:

This is used only in the inter ECU communication. When COM Tx timeout expired for the last positively accepted **Rte_Write** or **Rte_Send**, then the **Rte_Feedback API** returns with an **RTE_FBK_TIMEOUT**.

> RTE_FBK_TRANSMIT_ACK:

In case of inter or intra ECU communication, if a successful transmission acknowledgement is received from the COM for **Rte_Write** or **Rte_Send**, then the **Rte_Feedback API** returns with **RTE_FBK_TRANSMIT_ACK**. Even for queue overflow the same value will be returned.

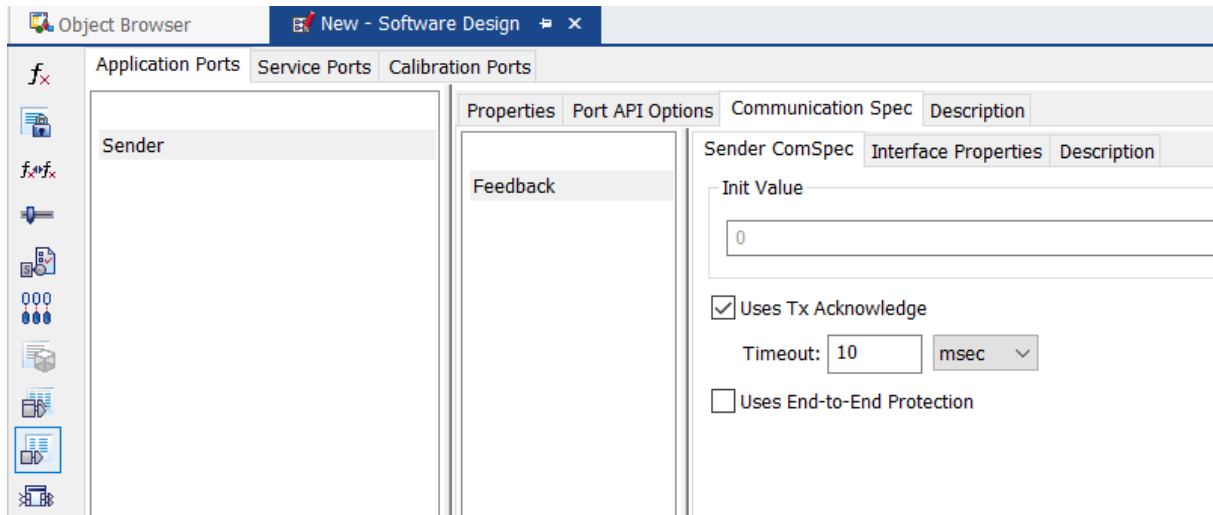
> RTE_FBK_NO_DATA:

When there is no acknowledgement or error notification from COM or wait point timeout expired for **Rte_Write** or **Rte_Send**, then the **Rte_Feedback API** returns with an **RTE_FBK_NO_DATA**. This state indicates that the Rte is still waiting for the outstanding TxConfirmation of the signal.

7 How to configure the RTE_Feedback API?

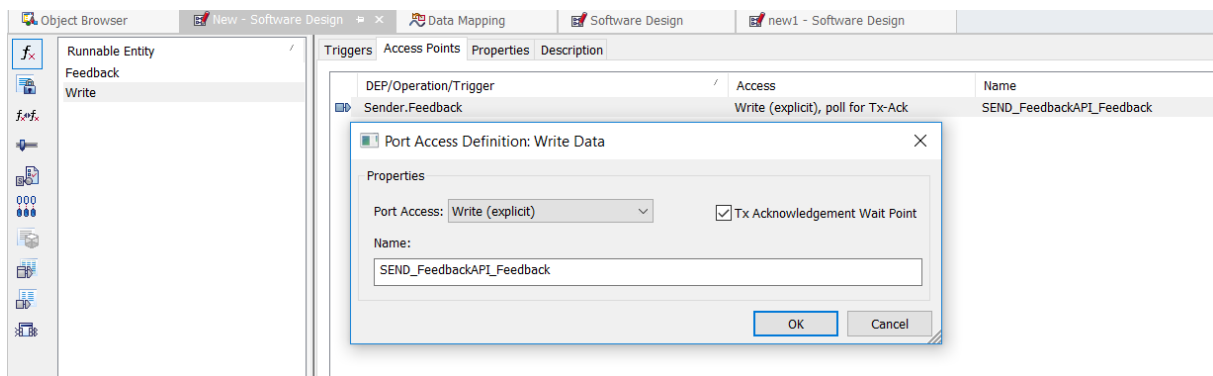
7.1 Port Configuration:

Rte_Feedback API should be configured as shown in the screenshot below. To generate the **Rte_Feedback API**, set the checkbox **Uses Tx Acknowledge** and configure the **Timeout** value if a wait point is required. (see 3)



7.2 Runnable Configuration for Wait Point:

A wait point should be configured in the access point of the runnable which calls the **Rte_Write API**. The access point should be configured as **Write (explicit)** and enable the checkbox **Tx Acknowledgement Wait Point**.

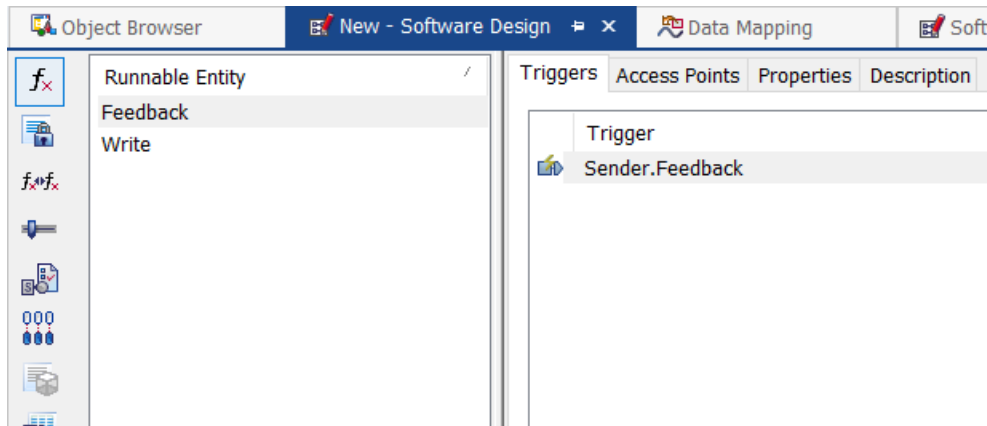


In this case (**Write (explicit), poll for Tx-Ack**) **Rte_Feedback** also belongs to the same runnable of **Rte_Write** or **Rte_Send API**.

7.3 Different Runnable for Feedback API:

To configure a different runnable for **Rte_Feedback API**.

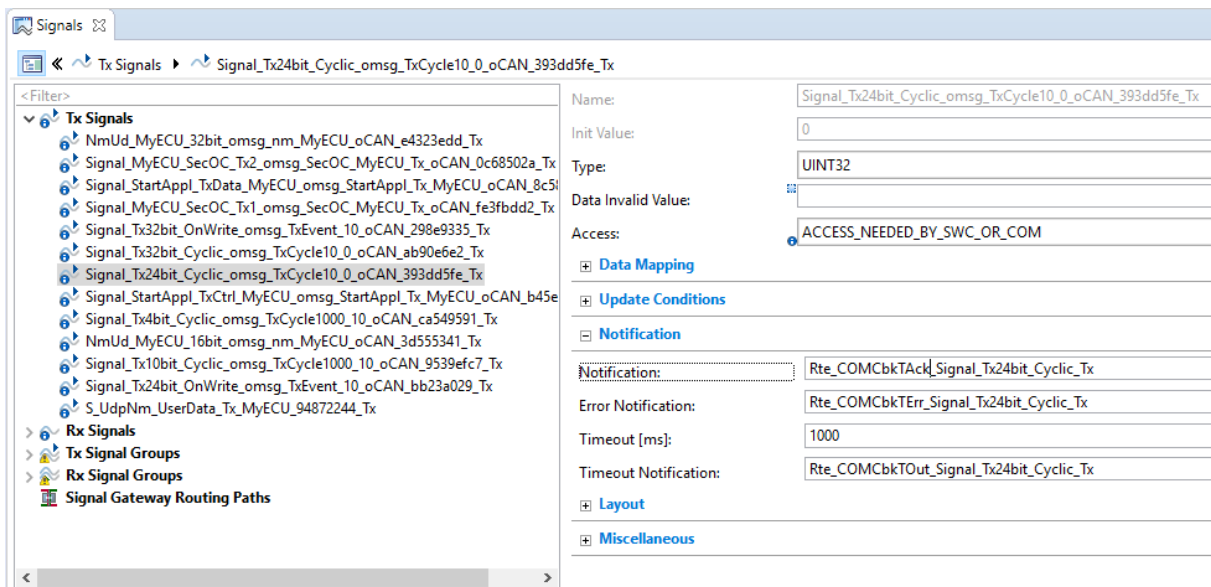
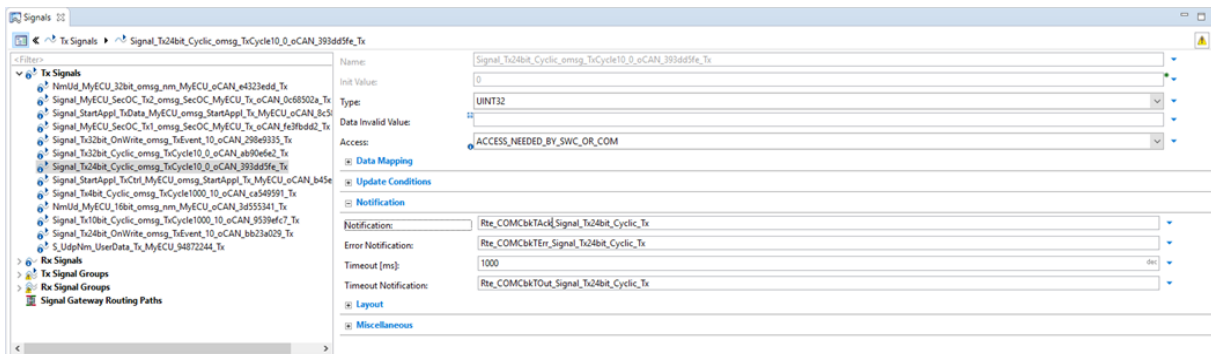
- > Create the runnable (here: feedback) and configure the trigger point as **Data Send Completed Event**.



> Map this runnable to valid task in the DaVinci Configurator Pro.

7.4 COM Configuration (Inter ECU Communication)

If the configurations above are done properly, Com signal notification will be done automatically, i.e. COM Error, COM Timeout and COM Notification should be configured automatically by the Rte calculation phase. Please find the screenshot below for your reference.



8 The Rte Feedback State gets stuck at RTE_E_NO_DATA – is this an Error?

The transition from **RTE_E_NO_DATA** which is a kind of waiting for **TxConfirmation-Or-Error** is not granted in all cases. There is a leak in the specification of BSW Module Com. In one certain case it is not specified to call the **Com_CbkTxErr** which then does not set the Rte_Feedback state back to **RTE_E_COM_STOPPED**. **SWS_Com_00491** is specifying the **Com_CbkTxErr** as “It is called in case the transmission is not possible because the corresponding I-PDU group is stopped”.

So if Com already called the **PduR_Transmit API** and this returned **E_OK** and then the Ipdu Group is stopped, then the **Com_CbkTxErr** is called → **Rte_Feedback** is set to **RTE_E_COM_STOPPED**. But when the Ipdu group is stopped in-between the **E_OK** from **Com_SendSignal[Group]()** and the **PduR_Transmit-Call** would return an **E_OK** (but the **PduR_Transmit-Call** was not performed yet), then the **Com_CbkTxErr** is not called and the **Rte_Feedback** state **RTE_E_NO_DATA** will never change until next call to **Rte_Start**.

Rationale: If **PduR_Transmit** returns **E_NOT_OK**, Com will retry the transmission on the next **Com_MainFunctionTx** call, so this is not a reason to call the **Com_CbkTxErr** function.

If an application state machine is dependent on the leakless transition out of this **RTE_E_NO_DATA** state there is a certain probability that this state machine gets stuck.



Workaround

Since the **Rte_Feedback** variable is only used for exactly this purpose, it is possible in certain cases to directly set this flag to **RTE_E_COM_STOPPED** as soon as the Ipdu group of Com is stopped, but this has to be handled with care, best is to contact the Vector Support in such a case.

9 Contacts

For support related questions please address to the support contact for your country
<https://www.vector.com/int/en/company/contacts/support-contact/>.